해태

Haetae

An RPKI validator that tells right from wrong

The haetae is a mythical Korean beast that judges right from wrong.
This Haetae judges whether the routes a router receives are right.



2026-11-11 · 0.3.0 · in trial operation · open source (ISC)

01 At a glance

An independent second check on what routers trust

0

difference from rpki-client

VRPs, ASPAs and router keys on the same input

~40 s

full validation

about 570,000 objects (rpki-client: 16 min)

0

external libraries

Go standard library only

1

binary

fetch, validate, RTR and API in one

Validates every standard RPKI object (ROA, ASPA, BGPsec keys and more: 9 types)

Serves RTR v0 / v1 / v2 to routers, an HTTP API to systems

Runs a resident daemon on a 10-minute cycle, checked daily against Routinator and rpki-client

RPKI: may this AS originate this route?

Resource holder

Signs a ROA ("this prefix is originated by this AS") for addresses from its RIR and publishes it

RP (Haetae)

Fetches every repository worldwide, checks signatures, validity and resources, and builds the VRP list

Router

Compares BGP routes with the VRPs and accepts or drops them by policy

Valid

A VRP matches the AS and length

accept

Invalid

A VRP covers it, but the AS or length differs

drop (stops hijacks and typos)

NotFound

No VRP covers it

accept as today

VRP = (AS, prefix, max length). ASPA uses customer-provider relations between ASes to stop route leaks; BGPsec keys verify path signatures

03 Why

A single RP mistake becomes a routing outage

Routers drop routes exactly as the RP says

- An RP bug or failure turns good routes Invalid, and they disappear
- Two RPs from different code bases expose each other's problems
- Yet few public RPs are fit for production (right)

Haetae is a new implementation that shares no code with existing RPs

Routinator	Rust · NLnet Labs	active
rpki-client	C · OpenBSD	active
FORT	C · NIC.mx	active
rpki-prover	Haskell · individual	active
OctoRPKI	Go · Cloudflare	archived
RIPE NCC Validator	Java · RIPE NCC	ended

Fetch, judge, serve: one run every 10 minutes

1
Fetch

- RRDP first, rsync fallback
- 32 repositories at once
- 5 min per repo, 20 min per run

2
Validate

- TAL → TA → CA → Manifest
- signatures, hashes, CRLs, dates
- resources within parent (RFC 3779)

3
Build

- VRPs, ASPAs, router keys
- SLURM local exceptions
- serial and deltas

4
Serve

- RTR v0 / v1 / v2
- HTTP API (queries, deltas)
- JSON and CSV files

If validation fails it keeps serving the last good result, and upgrades never drop router sessions

RFC 6487 · 3779 · 6488 · 9286 · 9582 · 8182 · 8210 · 8416 · 8630 · 9691

05 Objects

Every standard object in the repositories is validated

~570,000

objects validated (2026-11)

Type, URI, hash, validity, issuing CA, status and a content summary of every object, queryable as JSON

CA certificate
RFC 6487 · 3779
56,164

CRL
RFC 6487
56,078

Manifest
RFC 9286
56,078

ROA → VRP
RFC 9582
400,187

ASPA
draft aspa-profile
4,059

BGPsec router key
RFC 8209
3

GBR contact
RFC 6493
1

SPL
draft rpki-prefixlist
0 (none published)

TAK
RFC 9691
0 (none published)

Non-standard extensions (private or test objects) are only counted · RSC (RFC 9323) has a single-file check mode

06 Accuracy

Same input, zero difference from rpki-client

0

same cache, same validation time

- 1,029,840 VRPs, 3,904 ASPAs and router keys all match (fixed snapshot, ~570,000 objects)
- Also identical when rpki-client validates the cache Haetae fetched itself
- Re-checked against this snapshot after every code change

Daily comparison (2026-11-10)

Haetae	**1,033,062**	
Routinator	**1,033,060**	2 from Haetae
rpki-client	**1,033,047**	17 from Routinator

The remaining difference is ROAs that changed between fetch times (minutes apart). It is smaller than the difference between the other two RPs

07 Speed

16 minutes of validation, now about 40 seconds



Offline validation of the same cache (~570,000 objects) on the same server, each run alone · rpki-client 9.0 = Ubuntu 24.04 package · Haetae spreads validation over many cores

~1 min
one fetch + validation run (10-minute cycle)

~2 s
RTR pause during an upgrade (sessions stay up)

~5 GB
resident memory (peak during a run)

08 Standards

No TA constraints: the RFCs as written

Ubuntu 24.04 package file (2024-04)

2,773 VRPs · 59 ASPAs

valid objects rejected

After switching to the latest file (2026-07)

0 difference

same result as Haetae

rpki-client filters objects with per-RIR resource files (.constraints)

- Not in any RFC; specific to rpki-client, the files are made by its developers and shipped with it
- When a file lags IANA allocations, valid objects for newly allocated resources are rejected (above: 2410::/12, allocated to APNIC in 2024-11, is missing)
- Haetae does not use them (nor do Routinator or FORT) · RIR TAs holding all resources is standard practice

Measured 2026-10 · same data, same rpki-client 9.0, only the constraints file changed · difference = compared with validation without constraints

Routers get RTR v0, v1 and v2

v0

RFC 6810

down to the oldest routers (VRPs)

v1

RFC 8210

adds BGPsec router keys

v2

draft 8210bis

adds ASPA (route leak prevention)

Deltas	Serial Notify sends only changes · session and serial survive restarts and upgrades, so routers never reload everything
Negotiation	Answers at the router's version, stepping down v2 → v1 → v0
Access	allowed ranges, per-address connection limits, first-query / idle timeouts + optional TCP-MD5, TCP-AO, TLS, SSH

TCP 3323 (de facto standard port) · running with a Cisco IOS XR test router · full RTR data = JSON, checked with rtrdump and our own tool

Systems get an HTTP API

Routinator-compatible

- /api/v1/validity (route validity)
- /json-delta (changes only)
- /jsonext (source ROA of each VRP, optional)

rpki-client-compatible

- /json (VRPs, ASPAs, router keys)
- /csv
- same shape, so comparison tools read it as is

Haetae only

- AS and prefix lookup
- ASPA path verification (upstream / downstream)
- object and CA browsing, expiring soon
- unsafe VRPs, run history
- /healthz, Prometheus metrics

Two tiers by source: the management network (internal) gets every path, the Internet (public) only queries · public clients get per-address connection and rate limits, and internal data such as router addresses is hidden

11 Safety

Trust no input, stay steady through failure

Input is untrusted

- limits on size, count, depth and time
- huge RRDP elements stopped before memory
- https only, no path escapes
- one bad object fails only itself

When fetching fails

- bad manifest: last good copy (RFC 9286)
- network error: previous data, dead repositories retried less often
- deltas applied only after all checks
- run deadline: late repositories use previous data

When results look wrong

- not applied if any TA fails
- warning on a large VRP drop (optional hold)
- VRPs overlapping rejected CAs marked unsafe
- alarm after 1 hour without update

Dedicated non-root account · systemd sandboxing · 6 parsers fuzzed 3 hours each (~200 million runs, 0 issues) · two full code reviews (2026-11)

One binary, one config file, no downtime

One config file

every option is a config key · command line wins · unknown keys or bad values stop startup

Change without stopping

systemctl reload · all new values checked, then applied at once; on error nothing changes

Built-in TALs

official TALs of the 5 RIRs inside the binary · no dependence on other packages

Seamless upgrade

the same process becomes the new version · router sessions stay up, ~2 s pause

Monitoring

/healthz (200 when healthy, else 503) · Prometheus · systemd watchdog

Access control

separate allow lists for RTR and HTTP · HTTP in internal / public tiers

download a linux amd64 or arm64 binary and run it (no dependencies) · systemd service · manual and API docs included

What changed from 0.2.0 to 0.3.0

Security

- DER count, RRDP element size limits
- public HTTP: per-address limits, write timeouts
- failed TLS handshake frees its slot
- stricter systemd sandbox
- own router prefixes get their own limit (rtr.trusted)

Stability

- repository cap per TA, unused folder cleanup, disk check
- run deadline (20 min)
- publication point memory cap
- one bad object fails only itself

Operations

- warning on a TAK next-key notice
- slow repositories in the log
- deltas and RTR data in stable order
- SSH transport passes full responses
- linux amd64 · arm64 binaries released

All findings of the 2026-11 full code review fixed (5 high, 10 medium) · more parser fuzzing · still zero difference from rpki-client on the same input

Next: parallel validation, then production

Done · Published · Now · Next

Features

- all standard objects
- RTR v0 / v1 / v2, transport security
- ASPA path verification, RSC
- config, reload, built-in TALs

v0.1 → v0.3.0

- GitHub, ISC license
- code review fixes
- amd64 · arm64 binaries

Trial operation

- resident, 10-minute cycle
- daily result comparison
- test router, public test access

Parallel validation

- weeks alongside existing RPs
- RTR security and ASPA support per router vendor
- production decision

해태

Haetae

Independent code · same results · steady operation

An RPKI Relying Party built on the Go standard library alone · open source, ISC license

https://github.com/yunsog/haetae